Pair Trading with a Continuous Sizing Strategy

Robert Yang

Abstract

Pairs trading is a trading strategy first used in the 1980s performed on two related securities. When the spread (which can have different mathematical definitions depending on the model) widens, the outperforming security is shorted and the underperforming security is longed. The model effectively bets that the spread between the two securities will go back down. In this paper, we simulate a pair trading strategy performed on KO and PEP (Coca Cola and Pepsi) trained on data from 2004 to 2013 and tested on data from 2014 to 2022.

1 Examination of Data

In this section, we will introduce some pair trading theory and test that theory on KO and PEP to see if they are a good target for pair trading.

1.1 Cointegration test

Usually, pair trading is performed on securities which have a high degree of *cointegration*, which means a linear combination of their time series is relatively stationary. A stationary time series regularly goes back to or hovers around a specific value. For trading purposes, this is useful because this linear combination is predictable; when it's below the mean value, we can bet that it will eventually increase and vice versa. This allows us to make money with trades because it's easy to set up our positions to profit from this: simply a long on the underperforming stock and a short on the outperforming one, as is done for pair trading.

Surprisingly, KO and PEP do not have high cointegration (nor do their logs), as seen in the image below.

```
In [93]:  _, p_value, _ = coint(cc_train["Close"], pp_train["Close"])
          _, p_value_log, _ = coint(np.log(cc_train["Close"]), np.log(pp_train["Close"]))
          print(p_value, p_value_log)

          0.48243239781833713 0.5447540635422857
```

Nonetheless, we will proceeed with our pair trading test. The basis for our model is more based on fundamentals, namely that Coca Cola and Pepsi are both companies that have been around for a while, they're clearly closely related, and their relationship will likely not suddenly change unlike newer sectors like tech.

2 Training

We will train our model using KO and PEP data between 2003 and 2013 (inclusive). For the purposes of our relatively simple model, training will just mean finding a mean and standard deviation of the spread.

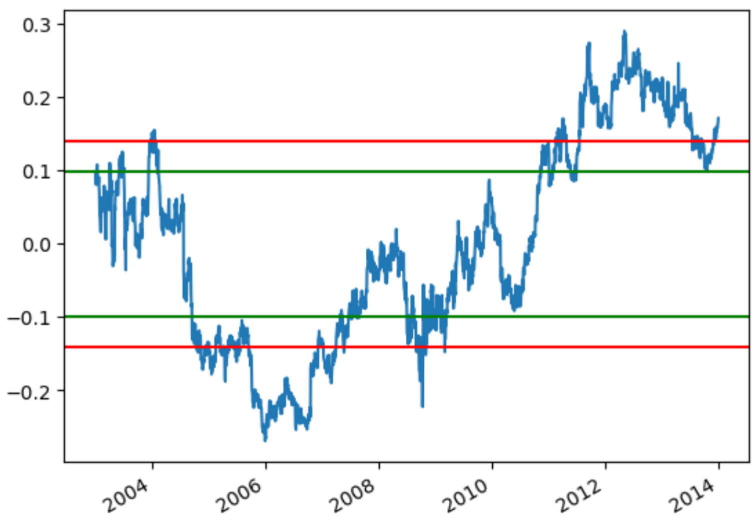
Submitted to 37th Conference on Neural Information Processing Systems (NeurIPS 2023). Do not distribute.

First, let's define our spread as $\log a - n \log b$, where a and b are the stock prices of KO and PEP respectively. n is a constant and we will define it to make the mean of the spread as close to zero as possible. We do this in the implementation below.

```
n = np.mean(np.log(cc_train["Close"]))/np.mean(np.log(pp_train["Close"]))
spread = np.log(cc_train["Close"]) - n*np.log(pp_train["Close"])
print(np.mean(spread), np.std(spread))
mid = np.mean(spread)
thresh = 0.7*np.std(spread)
spread.plot()
plt.axhline(y = np.std(spread), color = 'r', linestyle = '-')
plt.axhline(y = -np.std(spread), color = 'r', linestyle = '-')
plt.axhline(y = thresh, color = 'g', linestyle = '-')
plt.axhline(y = -thresh, color = 'g', linestyle = '-')
#(cc_train["Close"]/pp_train["Close"]).plot()
```

```
-6.520153716873752e-16 0.1406226163037814
```

```
5]: <matplotlib.lines.Line2D at 0x7fa2d13b7d00>
```



3 Testing our Strategy

3.1 Implementation

We will test our strategy using data from 2014 to 2022, inclusive. Our first strategy will create a buy/sell signal when the spread crosses below a z-score of -0.7 or above 0.7, and it ends the position when the spread goes back down to an absolute z-score of 0.2. Note that this z-score is based on the mean and standard deviation from the training data only. Below is the exact implementation.

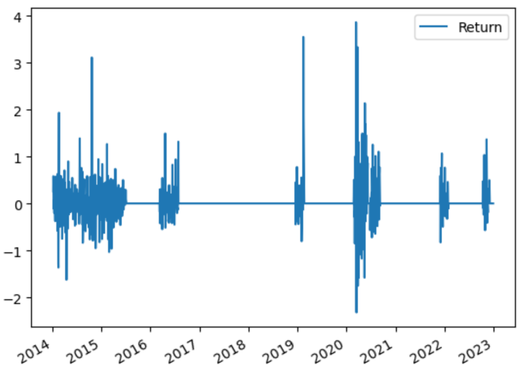
```
In [25]:
last_pos = 0
def make_pos(row, total_size):
    global last_pos
    high = [-total_size/2, total_size/2] # short KO, long PEP
    low = [total_size/2, -total_size/2] #  long KO, short PEP
    if last_pos == 1:
        if row < mid + 0.2 * np.std(spread):
            last_pos = 0
            return [0, 0]
        return high
    if last_pos == -1:
        if row > mid - 0.2 * np.std(spread):
            last_pos = 0
            return [0, 0]
        return low
    if row > mid+0.7*np.std(spread): # KO outperforming PEP
        last_pos = 1
        return high
    if row < mid-0.7*np.std(spread): # PEP outperforming KO
        last_pos = -1
        return low
    return [0, 0]
positions = spread_test.apply(make_pos, total_size = 100)
```

3.2 Results

Below is the graph of returns for our strategy and the total returns for all test years.

```
In [26]:  port_returns = pd.DataFrame({"Return":[]})
          for i in range(1, len(cc_returns)):
              port_returns.loc[cc_returns.index[i]] = positions.iloc[i-1][0] * cc_returns.iloc[i] + positions.iloc[i-1][1] * pp_returns
          port_returns.plot()
```

```
Out[26]: <AxesSubplot:>
```




```
In [27]:  sum(port_returns["Return"])
```

```
Out[27]: 35.95149803895159
```

Our strategy made less returns than longing both KO and PEP would've made.

```
#long both strategy
50*(cc_test.iloc[-1]["Close"]/cc_test.iloc[0]["Close"]-1) + 50*(pp_test.iloc[-1]["Close"]/pp_test.iloc[0]["Close"]-1)
```

```
88.24620550102381
```

However, our strategy had a very conservative position size of $100 at all times (where a short counts as a position size). Unlike longing both stocks, this doesn't take advantage of compound growth. In addition, since we always longed a stock and shorted another, the total amount of money we put in to make a position was $0, so we could increase our position sizes by much more than in the long both strategy.

Furthermore, our strategy achieved a much better Sharpe ratio, as seen below.

```
sharpe_ratio = np.mean(port_returns["Return"])/np.std(port_returns["Return"])
asr = sharpe_ratio*252**.5
print(sharpe_ratio, asr)
```

```
0.055190565142718605 0.8761230605084607
```

```
both = cc_returns["Return"] + pp_returns["Return"]
sharpe_ratio_long = np.mean(both)/np.std(both)
asr_long = sharpe_ratio_long*252**.5
print(sharpe_ratio_long, asr_long)
```

```
0.031387324764646014 0.49825833388123375
```

Pair trading is structured to be market neutral and minimize risk, so the Sharpe ratio is a fair indicator of the quality of a pair trading strategy.

4 Continuously Sized Position Strategy

In this section, we will test an alternative model, invented by the authors, that continuously sizes our position based on the spread.

4.1 Theory

We make a new position at the close of each day (by trading after-hours or right before the close) and our trade size will be proportional to the z-score of the spread.

The logic here is that our spread is mean reverting, so consider what happens if the spread follows a mean reversion sequence $a = (0, 0.1, 0.2, ..., 1, 0.9, 0.8, ..., 0)$, where each spread value is at the close of each day. Then our position sizes based on day i is proportional to a_i and we make money if a_{i+1} (the close price of the next day) is lower and lose money if it's higher (since we expect the positive z-score spread to fall back to the mean). Thus, our return for the position started on day i will be $c(a_{i+1} - a_i)a_i$ for some constant c. Based on this formula and the sequence, our return will be $0.1c(-0 - 0.1 - 0.2 - \ldots - 0.9 + 1 + 0.9 + \ldots + 0.1) = 0.1c(1)$. This is equivalent to if we had started our position when the spread had z-score 1 and ended it when the spread returned to the mean. Note that 1 was the arbitrary maximum of the sequence, so this strategy effectively gave the same return as if our previous strategy was magically able to detect when the spread time series would locally peak, which is the best time to start a position.

The issue is the spread clearly wouldn't follow such a nice sequence in practice, but the general idea still holds: because our position size has magnitude proportional to the spread z-score of the previous day, our positions will be slightly larger when the spread reverts to the mean than when the spread strays away from the mean, generating a net profit.

4.2 Implementation and Results

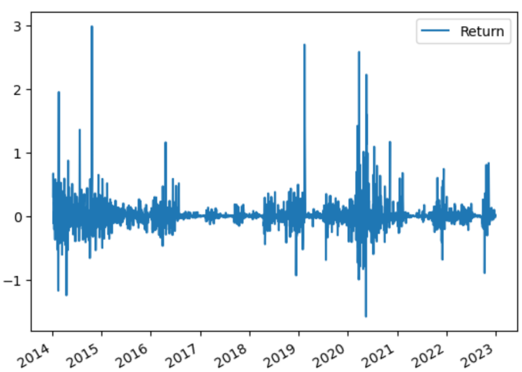
Below is my implementation and the returns graph.

```
In [87]:  def make_pos_continuous(row, size): #size is for one stdev away from the mean
              return size*row/np.std(spread)*np.array([-1/2, 1/2])
          positions_cont = spread_test.apply(make_pos_continuous, size = 100)
          positions_cont

Out[87]:  Date
          2014-01-02      [-60.04057456518083, 60.04057456518083]
          2014-01-03    [-57.801284808107496, 57.801284808107496]
          2014-01-06      [-55.98893441076321, 55.98893441076321]
          2014-01-07      [-52.91660273563627, 52.91660273563627]
          2014-01-08    [-49.754220161341124, 49.754220161341124]
                                          ...
          2022-12-23      [7.157144984285845, -7.157144984285845]
          2022-12-27        [6.25590357921615, -6.25590357921615]
          2022-12-28        [7.75338901461039, -7.75338901461039]
          2022-12-29      [5.995035854538126, -5.995035854538126]
          2022-12-30      [5.813813595796454, -5.813813595796454]
          Name: Close, Length: 2266, dtype: object
```

```
In [88]:  port_returns_cont = pd.DataFrame({"Return":[]})
          for i in range(1, len(cc_returns)):
              port_returns_cont.loc[cc_returns.index[i]] = positions_cont.iloc[i-1][0] * cc_returns.iloc[i] + positions_cont.iloc[i-1][
          port_returns_cont.plot()

Out[88]:  <AxesSubplot:>
```



And below is the total returns, average position size, and Sharpe ratio.

```
In [90]:  sum(port_returns_cont["Return"])

Out[90]:  29.58822793246253
```

```
In [92]:  # prints average position size on KO for the continuous strategy and the original
          print(np.mean([abs(i[0]) for i in positions_cont]), np.mean([abs(i[0]) for i in positions]))

          17.794058665387798 15.688437775816416
```

```
In [93]:  print(get_sharpe(port_returns_cont["Return"]))

          [0.058307647165710276, 0.9256052036026172]
```

Note that this strategy had lower returns than the original strategy while having a higher average position size, so the percentage return wasn't as great. However, this strategy achieved a slightly better Sharpe ratio.

In conclusion, this is an interesting strategy which seems like it can be optimized to perform quite well. It is more difficult to execute in practice because it needs to make new positions every day, however.

References

[1] Quantitative Finance & Algo Trading Blog by QuantInsti. (2023, March 16). Pairs trading basics: Correlation, cointegration and strategy. Quantitative Finance & Algo Trading Blog by QuantInsti. Retrieved May 7, 2023, from https://blog.quantinsti.com/pairs-trading-basics/

[2] KidQuant. (2021, May 5). Pairs-trading-with-python/pairstrading.ipynb at master · KidQuant/pairs-trading-with-python. GitHub. Retrieved May 7, 2023, from https://github.com/KidQuant/Pairs-Trading-With-Python/blob/master/PairsTrading.ipynb

[3] Ethan Johnson-Skinner, M. S. (2022, November 11). Creating and implementing a pairs trading strategy from scratch. Medium. Retrieved May 7, 2023, from https://medium.datadriveninvestor.com/creating-and-implementing-a-pairs-trading-strategy-from-scratch-658267bab249